

Universidad Nacional Autonoma de Mexico
Facultad de Ingenieria
Computer Engineering · Compilers

CompyFI LR(0)

Educational Compiler with LR(0) Parsing, SDT and
TAC

Team: 08
Group: 5
Semester: 2026-II
Package: `unam.fi.compilers.g5.08`
Students: 320198687
320301238
321332086
321132927
321132477

Contents

1	Introduction	1
2	General Objective	1
3	Specific Objectives	1
4	Project Overview	2
5	Theoretical Background	2
5.1	Compiler	2
5.2	Formal Languages and Automata	2
5.3	Lexical Analysis	3
5.4	Syntactic Analysis and LR(0)	3
5.5	Syntax-Directed Translation	3
5.6	Intermediate Code and TAC	3
6	Grammar Used	3
6.1	Augmented Grammar	3
6.2	Grammar Interpretation	4
6.3	Why It Is a Context-Free Grammar	4
6.4	Example of a Valid Input	4
7	Lexical Analysis	4
7.1	Recognized Tokens	4
7.2	Tokenization Example	5
7.3	Lexical Errors	5
8	LR(0) Parser	5
8.1	LR(0) Items	5
8.2	Closure Operation	6
8.3	Goto Operation	6
8.4	Canonical Collection of States	6
8.5	State Transitions	6
8.6	ACTION/GOTO Table	6
8.7	Parsing Trace	6
9	Parsing Result	7
10	Semantic Validation and Symbol Table	7
10.1	Semantic Validation	7
10.2	Semantic Rules Applied	7
10.3	Generated Symbol Table	7
10.4	Detected Semantic Errors	7
11	Syntax-Directed Translation (SDT)	8
11.1	place	8
11.2	newTemp()	8
11.3	gen()	8

11.4 SDT Rules Used as Explanation	8
12 TAC Generation	8
12.1 TAC for a Simple Statement	8
12.2 TAC for an If Statement	8
12.3 TAC for a While Statement	8
12.4 Cases Where TAC Is Not Generated	9
13 Backpatching as a Related Stage	9
14 Optimization as a Later Stage	9
15 Compiler Interface	9
15.1 Main View	9
15.2 Main Menu	10
15.3 Source Code Input	10
15.4 Compilation Process Visualization	10
16 Error Handler	10
16.0.1 Lexical Errors	10
16.0.2 Syntactic Errors	10
16.0.3 Semantic Errors	11
17 Test Cases	11
17.1 Valid Case	11
17.2 Case with Syntactic Error	11
17.3 Case with Lexical Error	11
17.4 Case with Semantic Error	11
17.5 Case with Empty Block	12
17.6 Case with Incomplete Condition	12
18 Result Analysis	12
19 System Architecture	12
19.1 Role of Each File	13
19.2 Execution Flow	13
20 Basic User Manual	13
21 Limitations	14
22 Future Work	14
23 Conclusions	14
A Project File Structure	15
B Predefined Examples in the Application	16
C Detailed Execution Instructions	16

1. Introduction

CompyFI LR(0) is an educational compiler-like system designed to support the study of the main topics of a Compilers course. The project does not attempt to implement an industrial compiler or the full C language; instead, it focuses on a reduced formal language so that the internal behavior of each compilation phase can be observed clearly.

The system integrates lexical analysis, LR(0) syntactic analysis, Syntax-Directed Translation (SDT) concepts, and elementary Three Address Code (TAC) generation. Its most important contribution is not only accepting or rejecting an input, but showing visually how each decision is made: the interface displays the token stream, grammar, LR(0) states, transitions, canonical collection, ACTION/GOTO table, parsing trace, SDT rules, and generated TAC.

The final version also includes theoretical sections related to the official Compilers syllabus: formal languages, lexical analysis, syntactic analysis, semantic analysis, intermediate code, backpatching, optimization, and the complete compilation flow. In this sense, CompyFI works both as a reduced front-end compiler and as a visual study tool.

The project was implemented applying only the theoretical concepts seen in class, without the use of external parsing libraries, which satisfies the extra-points criterion established in the final project specification.

2. General Objective

To develop a didactic web application that integrates lexical analysis, LR(0) parsing, basic Syntax-Directed Translation concepts, and elementary intermediate code generation, in order to facilitate the practical understanding of the compilation flow for a reduced language defined by a context-free grammar.

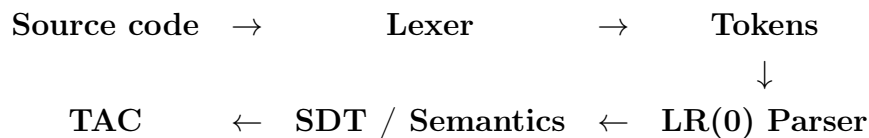
3. Specific Objectives

- Design a lexical analyzer capable of recognizing reserved words, identifiers, relational operators, punctuation symbols, and the end-of-input marker of the implemented language.
- Define a reduced context-free grammar that represents blocks, statement lists, simple statements, **if** statements, **while** statements, and relational conditions.
- Build the augmented grammar required by the LR(0) method.
- Generate the canonical collection of LR(0) items, including closure and goto operations.
- Construct the ACTION/GOTO table and use it in a stack-based shift-reduce parser.
- Display the parsing trace step by step, including stack contents, remaining input, and action taken.
- Present simplified SDT concepts using the attributes **place**, **newTemp()**, and **gen()**.

- Generate elementary TAC for simple statements and control structures such as `if` and `while`.
- Integrate all components into a local web interface with theoretical explanations and visual outputs.
- Implement an error handler that reports lexical, syntactic, and semantic errors with detailed information about the point of failure.

4. Project Overview

CompyFI follows a sequential compiler-like workflow. First, the source code is entered by the user in the web interface. The lexer converts the raw text into tokens. If no lexical error is detected, the LR(0) parser verifies whether the token sequence belongs to the language defined by the grammar. When syntactic analysis concludes successfully, the system executes the basic semantic validation and generates the TAC intermediate representation.



The interface is organized around a sidebar menu. The main view is the compiler view, where the user can edit the input code and inspect the analysis results. Additional views provide theoretical content and background topics related to the course syllabus.

5. Theoretical Background

5.1. Compiler

A compiler is a system that processes a program written in a formal language and transforms it into increasingly structured internal representations. In a complete compiler, the general process includes lexical analysis, syntactic analysis, semantic analysis, intermediate code generation, optimization, and target code generation [1]. In CompyFI, the scope is intentionally reduced and focuses on the front-end phases plus basic TAC generation.

5.2. Formal Languages and Automata

Formal languages provide the theoretical basis for defining what strings belong to a language. Concepts such as symbols, alphabets, strings, languages, grammars, terminals, non-terminals, productions, and start symbols are necessary to formally describe a programming language [2]. Regular expressions and finite automata are connected with lexical analysis because they describe and recognize lexical patterns [3, 4]. Context-free grammars are connected with syntactic analysis because they define hierarchical structures [5]. Pushdown automata provide the conceptual foundation for stack-based syntactic analysis [7].

5.3. Lexical Analysis

Lexical analysis transforms a sequence of characters into a sequence of tokens. This phase also removes whitespace and reports unknown characters when they do not belong to the accepted alphabet of the language [1].

5.4. Syntactic Analysis and LR(0)

Syntactic analysis verifies whether the token sequence follows the structural rules defined by a grammar. The LR(0) method is a bottom-up shift-reduce analysis strategy. It uses an augmented grammar, a canonical collection of LR(0) items, closure and goto operations, an ACTION/GOTO table, and a stack to decide whether to shift, reduce, accept, or report an error [6].

5.5. Syntax-Directed Translation

Syntax-Directed Translation (SDT) connects grammar productions with semantic actions. These actions may compute attributes, generate intermediate code, or prepare information for later phases. In CompyFI, SDT is presented through the attributes and helper functions `place`, `newTemp()`, and `gen()`.

5.6. Intermediate Code and TAC

Three Address Code (TAC) is an intermediate representation where each instruction is written in a simple form, usually with temporaries, labels, assignments, conditions, and jumps. TAC is useful because it separates syntactic recognition from optimization and target code generation [1].

6. Grammar Used

6.1. Augmented Grammar

The final version of CompyFI uses the following augmented context-free grammar:

No.	Production	Description
0	$P' \rightarrow P$	Augmented production
1	$P \rightarrow B$	Program is a block
2	$B \rightarrow \{W\}$	Block with statements
3	$B \rightarrow \{\}$	Empty block
4	$W \rightarrow W S$	Recursive statement list
5	$W \rightarrow S$	Single-statement list
6	$S \rightarrow \text{while} (C) B$	While statement
7	$S \rightarrow \text{if} (C) B$	If statement
8	$S \rightarrow i ;$	Simple statement
9	$C \rightarrow i < i$	Relational condition

The production $P' \rightarrow P$ is the augmented initial production. It does not change the recognized language, but it gives the parser a formal acceptance condition: the input is accepted when the completed item $P' \rightarrow P \cdot$ is reached with the end marker $\$$.

6.2. Grammar Interpretation

The grammar describes a reduced block-based language. A complete program is modeled as a block. A block can be empty or contain a list of statements. A statement can be a `while` structure, an `if` structure, or a simple identifier statement followed by a semicolon. A condition has the fixed form `i < i`.

6.3. Why It Is a Context-Free Grammar

This grammar is context-free because each production has exactly one non-terminal on the left-hand side. For example, $B \rightarrow \{W\}$, $S \rightarrow i ;$, and $C \rightarrow i < i$ all rewrite a single non-terminal independently of the surrounding symbols. This property matches the formal definition of a context-free grammar, where productions have the general form $A \rightarrow \alpha$, with A as a non-terminal and α as a sequence of terminals and non-terminals [5].

It is suitable for LR(0) because the states of the canonical collection allow deciding between shift, reduce, accept, or error without a lookahead symbol, and the resulting ACTION/GOTO table has no shift/reduce or reduce/reduce conflicts.

6.4. Example of a Valid Input

```
1 {  
2     i;  
3     while ( i < i ) {  
4         i;  
5     }  
6     if ( i < i ) { }  
7 }
```

This input is valid because it starts as a block, contains valid statements, uses conditions of the form `i < i`, and correctly closes all braces.

7. Lexical Analysis

7.1. Recognized Tokens

The lexer recognizes a reduced set of symbols according to the current grammar. Its output is handled as an ordered list of tuples (`type`, `lexeme`).

Token type	Description	Example
<code>while</code>	Reserved word for loops	<code>while</code>
<code>if</code>	Reserved word for conditionals	<code>if</code>
<code>i</code>	Generic identifier	<code>x</code> , <code>count</code>
<code><</code>	Relational operator	<code><</code>
<code>(</code>	Opening parenthesis	<code>(</code>
<code>)</code>	Closing parenthesis	<code>)</code>
<code>{</code>	Opening brace	<code>{</code>
<code>}</code>	Closing brace	<code>}</code>
<code>;</code>	Statement terminator	<code>;</code>
<code>\$</code>	End-of-input marker	<code>\$</code>
UNKNOWN	Unrecognized character	<code>@</code> , <code>#</code>

7.2. Tokenization Example

For an input such as `{ i; if ( i < i ) { } }`, the generated token sequence is:

```
{ i ; if ( i < i ) { } } $
```

Internally, each element is stored together with its original lexeme. A concrete identifier such as `counter` is classified as `i`, since this version of the language does not distinguish between specific identifier names during syntactic analysis.

7.3. Lexical Errors

The lexer reports an error when it encounters a character that cannot be classified by any of the defined patterns. The symbol is added to the sequence as UNKNOWN and compiler execution stops before syntactic analysis. If lexical errors exist, the system returns `parsing_success = False` and explicitly indicates that semantic validation and TAC generation will not be executed.

8. LR(0) Parser

The LR(0) parser is the core of CompyFI's syntactic analysis. Its responsibility is to determine whether the token sequence produced by the lexer can be reduced to the grammar's start symbol.

8.1. LR(0) Items

An LR(0) item is a grammar production with a dot that marks how much of the production has already been recognized by the parser. The dot is not part of the language; its purpose is to represent the progress of the analysis within a rule. For example:

```
C -> i < . i
```

This means the parser has already recognized `i <` and still expects another identifier to complete the condition. Items are the conceptual foundation of the entire bottom-up analysis.

8.2. Closure Operation

The closure operation expands a set of items whenever the dot appears before a non-terminal, incorporating the productions for that symbol with the dot at the beginning. This operation builds complete LR(0) states.

8.3. Goto Operation

The goto operation moves the dot over a given symbol and then applies closure to the result. This operation creates transitions between LR(0) states.

8.4. Canonical Collection of States

The canonical collection is the set of all LR(0) states generated from the augmented grammar. Each state represents a possible syntactic context of the parser. In the interface, this collection is displayed as separate state cards so the user can observe each set of active items and its outgoing transitions.

8.5. State Transitions

Transitions indicate how the parser moves from one state to another after recognizing a terminal or non-terminal. Transitions over terminals are associated with shift actions, while transitions over non-terminals are used in the GOTO part of the table after reductions.

8.6. ACTION/GOTO Table

The ACTION/GOTO table determines the parser operation for each state and input symbol.

Notation	Meaning
sN	Shift and move to state N
rN	Reduce using production N
acc	Accept the input
Empty cell	Report syntactic error

When a cell is empty, the analysis stops and a syntactic error is produced. The system stores the current state, the received token, the concrete lexeme, and the list of expected symbols, and presents that diagnostic in the result section.

8.7. Parsing Trace

The parsing trace records the complete shift-reduce process step by step. Each row shows the stack contents, the remaining input, and the action applied at that instant. This view converts the shift-reduce procedure into an observable sequence. If the string belongs to the language, the trace concludes with **acc**. If not, it ends with **error** and precisely indicates the point where the input stopped matching the grammar.

9. Parsing Result

The result section concentrates the syntactic verdict of the compiler. When the string is valid, the interface shows a success message and confirms that the ACTION/GOTO table accepted the complete input. From that point, the following phases are enabled: semantic validation and TAC generation.

When the string does not belong to the language, the result preserves the information needed to explain the error precisely: current state, received token, received lexeme, expected symbols, and a message of the form `No action in ACTION[state, token]`. The interface stops semantic analysis and TAC at that point.

10. Semantic Validation and Symbol Table

10.1. Semantic Validation

Semantic validation is executed only after a successful syntactic acceptance. The function `validate_types(lexer_tokens)` extracts identifiers from the token stream, registers them, and builds a minimal symbol table. If validation succeeds, the system emits a message summarizing the semantic consistency of the program.

10.2. Semantic Rules Applied

The semantic rules are based on the SDT concepts presented in the interface. The attribute `place` stores the result of a production. The function `newTemp()` creates temporary variables. The function `gen()` emits an intermediate code instruction. The simplified rules used as explanation are:

```
P'.place = P.place
P.place = B.place
B.place = W.place
B.place = empty
W.place = S.place
S.place = newTemp(); gen(S.place = id.lexeme)
C.place = newTemp(); gen(C.place = id.left < id.right)
```

10.3. Generated Symbol Table

The symbol table is presented with the columns Name, Category, Type, Value, Declared, and Used. For a valid input with identifiers, the table registers each identifier recognized in statements and conditions with category `identifier`, type `id`, and declaration and use flags set to true.

10.4. Detected Semantic Errors

Semantic errors in CompyFI appear when the input has already been accepted by the parser but does not contain valid identifiers to build the symbol table. When semantics fails, the system flow stops before TAC and emits the corresponding message indicating that syntactic analysis succeeded but semantic validation failed.

11. Syntax-Directed Translation (SDT)

11.1. place

The attribute `place` stores where the result of a production or expression is located. In a translation process, it works as a placeholder that remembers the result associated with the right-hand side of a production.

11.2. newTemp()

The function `newTemp()` creates temporary variables. These temporaries are useful when a condition or expression must be stored before generating the next TAC instruction.

11.3. gen()

The function `gen()` emits an intermediate code instruction. In the project, this concept is used to explain how recognized structures can be translated into TAC instructions.

11.4. SDT Rules Used as Explanation

The simplified rules describe the intended translation behavior and connect the grammar with the idea of attributes and intermediate code generation. They do not replace the actual validation, but serve an explanatory function that links syntactic acceptance with the preparation of intermediate code.

12. TAC Generation

TAC generation is executed only when syntactic analysis and semantic validation both conclude successfully.

12.1. TAC for a Simple Statement

A simple statement `i;` is represented using a temporary:

```
1  t1 = i
```

12.2. TAC for an If Statement

An `if` statement with a condition of the form `i < i` is represented with a condition temporary and a false jump:

```
1  t1 = i < i
2  ifFalse t1 goto L1
3  // if body
4  L1:
```

12.3. TAC for a While Statement

A `while` statement uses labels to represent repetition:

```
1 L1:
2 t1 = i < i
3 ifFalse t1 goto L2
4 // while body
5 goto L1
6 L2:
```

12.4. Cases Where TAC Is Not Generated

TAC is not generated when any previous phase of the compiler fails:

- **Lexical error:** If the input contains an unrecognized symbol, the flow stops before syntactic analysis.
- **Syntactic error:** If the parser rejects the input, execution stops before semantics.
- **Semantic error:** If semantic validation fails, the generator does not produce TAC instructions.

13. Backpatching as a Related Stage

Backpatching is related to intermediate code generation. It is used when the compiler generates a jump instruction before knowing its final destination. The destination is left pending and filled in later when the correct label is known [1].

In CompyFI, labels are generated directly in a simplified way. Therefore, backpatching is presented as theoretical context rather than a fully implemented module. This is appropriate because the grammar is small, but the topic helps explain how a future version could handle more complex control flows using pending jump lists.

14. Optimization as a Later Stage

Optimization improves intermediate or target code without changing the meaning of the program. Machine-independent techniques include dead code elimination, constant propagation, constant folding, common subexpression elimination, strength reduction, and algebraic simplification [1].

The current version of CompyFI does not implement a real optimization phase. Optimization is included in the theoretical interface sections to connect the project with the complete compilation flow.

15. Compiler Interface

15.1. Main View

The interface is one of the most important components of the final version. It is generated through Python and served locally. The compiler view is divided into three main areas: predefined examples, source code, and LR(0) results. The result panel uses tabs to show: Result, Grammar, Tokens, States, Transitions, Canonical Collection, ACTION/GOTO, Trace, TAC, and SDT.

15.2. Main Menu

The sidebar menu organizes the general navigation of CompyFI. From it, the user can access the compiler and complementary sections: Course Syllabus, Lexical Analysis, Syntactic Analysis, Semantic Analysis and SDT, Intermediate Code and TAC, Backpatching, Code Optimization, Complete Flow, Background, and Creators.

15.3. Source Code Input

The local server handles GET requests to display the interface and POST requests to receive the content of the text area. The user can work with predefined examples or write code manually. The input logic does not depend on external files during normal web interaction.

15.4. Compilation Process Visualization

The platform does not reduce execution to a final verdict. Each analysis phase can be reviewed separately. Processing follows a fixed sequence: first tokenization, then automaton construction and parser execution, and only if syntactic analysis succeeds are semantic validation and TAC generation enabled.

16. Error Handler

CompyFI implements an error handler that acts at three levels of the compilation flow:

16.0.1. Lexical Errors

When the lexer encounters a character that does not belong to the language alphabet, it classifies it as `UNKNOWN`, reports it in the error list, and stops the process before syntactic analysis. The interface clearly indicates that syntactic analysis, semantics, and TAC were not executed due to the lexical error.

16.0.2. Syntactic Errors

When the parser finds no valid action in the `ACTION` table for the current state and token combination, it builds an `error_info` object with the following fields:

- **state:** Parser state at the moment of the error.
- **token:** Type of the token that caused the error.
- **lexeme:** Concrete text of the token.
- **expected:** List of valid tokens in that state.
- **message:** Description such as `No action in ACTION[state, token]`.

This diagnostic allows locating the problem within the automaton and linking it with the grammar, the table, and the trace.

16.0.3. Semantic Errors

When semantic validation fails, the system emits a semantic error message, generates an empty symbol table, and does not produce TAC. The interface clearly distinguishes between syntactic success and semantic success, showing that a string can be structurally well-formed but fail at the meaning level.

17. Test Cases

17.1. Valid Case

Input: { i; while (i < i) { i; } if (i < i) { } }

Expected result: Accepted. The lexer correctly tokenizes the entire input. The parser accepts it after a complete sequence of shifts and reduces concluding in acc. Semantic validation registers the identifiers and builds the symbol table. The TAC generator produces instructions for the simple statement, the while loop, and the if structure.

Reason: Satisfies $B \rightarrow \{W\}$ and all statements are valid.

17.2. Case with Syntactic Error

Input: { i } (missing semicolon)

Expected result: Syntactic error. The lexer produces valid tokens. However, the parser finds that after recognizing i, it expects ; but encounters }. The ACTION table contains no action in that state for that token.

Reason: The production $S \rightarrow i$; requires a semicolon unconditionally.

17.3. Case with Lexical Error

Input: { i; @ }

Expected result: Lexical error. The character @ does not belong to the language alphabet. The lexer classifies it as UNKNOWN and stops the process before syntactic analysis.

Reason: Characters not belonging to the alphabet {while, if, i, <, (,), {, }, ;} are reported as unknown.

17.4. Case with Semantic Error

Input: A syntactically valid block without sufficient identifiers to build the symbol table.

Expected result: The parser accepts the input, but semantic validation fails when no valid identifiers are found. The symbol table remains empty and the system emits the message indicating that syntactic analysis succeeded but semantic validation failed. No TAC is generated.

Reason: The semantic phase requires recognizable identifiers in the token stream to build the symbol table.

17.5. Case with Empty Block

Input: `{ }`

Expected result: Accepted. The production $B \rightarrow \{\}$ allows a block without statements. No TAC is generated because the block contains no translatable structures.

Reason: The grammar explicitly includes the empty block as a valid input.

17.6. Case with Incomplete Condition

Input: `{ while ( i < ) { i; } }`

Expected result: Syntactic error. The production $C \rightarrow i < i$ requires an identifier on both sides of the relational operator. If the second identifier is missing, the parser cannot complete the condition.

18. Result Analysis

The valid case demonstrates the complete system flow: the lexer recognizes all symbols, the parser accepts the token sequence, and the TAC tab shows intermediate instructions for statements and control structures.

The empty block is accepted because the grammar explicitly includes $B \rightarrow \{\}$. This case is important because it proves that the parser can accept a block without statements.

The missing semicolon case fails syntactically because the grammar only recognizes a simple statement through $S \rightarrow i ;$. If the semicolon is absent, the ACTION table has no valid action for the current state and input symbol.

The incomplete condition fails because $C \rightarrow i < i$ requires an identifier on both sides of the relational operator. If the second identifier is missing, the parser cannot complete the condition.

The missing brace case fails because blocks must close properly. This error shows that the parser enforces complete structural rules and not only isolated token patterns.

19. System Architecture

CompyFI follows a modular organization that clearly separates the responsibilities of the compiler and the web interface. The system is divided into two layers:

- **Compiler core:** `lexer.py`, `parser_lr0.py`, `sdt.py`, `tac.py`.
- **Presentation layer:** `ui.py` and `main.py`.

19.1. Role of Each File

File	Responsibility
<code>main.py</code>	Entry point. Starts the local HTTP server, handles GET and POST routes, and delegates content construction to <code>ui.py</code> .
<code>ui.py</code>	Concentrates the presentation logic. Generates all HTML views, executes the complete compilation flow via <code>process_code()</code> , and builds each output tab.
<code>lexer.py</code>	Implements lexical analysis. Scans the input character by character and produces an ordered list of <code>(type, lexeme)</code> tuples.
<code>parser_lr0.py</code>	Defines the grammar, builds the LR(0) canonical collection, generates transitions, constructs the ACTION/GOTO table, and executes the shift-reduce procedure with a stack.
<code>sdt.py</code>	Implements basic semantic validation and symbol table construction.
<code>tac.py</code>	Produces elementary TAC intermediate code for the structures recognized by the grammar.
<code>index.html</code>	Static HTML shell that serves as the visual entry point before the Python server processes requests.
<code>multimedia/</code>	Folder with visual interface resources (mascot, images).

19.2. Execution Flow

The execution flow starts when the application is opened in the browser and the user accesses the compiler view. When the input is submitted, `main.py` receives the content and calls `ui.py`, where the complete process is executed:

1. **Lexical analysis:** `lexer.py` transforms the source code into tokens. If there is a lexical error, the flow stops.
2. **Syntactic analysis:** `parser_lr0.py` builds the automaton, the LR(0) table, and runs the parser. If it fails, the flow stops.
3. **Semantic validation:** `sdt.py` registers identifiers and builds the symbol table. If it fails, the flow stops.
4. **TAC generation:** `tac.py` produces the intermediate code.
5. **Visualization:** `ui.py` organizes all outputs into tabs within the interface.

20. Basic User Manual

To run the project, the user must open the project folder and execute:

```
1 python main.py
```

The console prints a local URL similar to:

```
http://127.0.0.1:8000/index.html
```


After opening the URL, the user can select a predefined example or write a custom input in the code editor. The available examples are: valid block with while and if, empty block, missing semicolon, incomplete condition, and missing closing brace.

Requirements: Python 3.10 or higher installed with the *Add to PATH* option enabled.

21. Limitations

- The supported language is reduced. It does not implement real declarations, assignments, arithmetic expressions, function parameters, scopes, types beyond generic identifiers, **else**, **return**, arrays, or target code generation.
- The SDT section is simplified and mainly explanatory. It does not implement a complete semantic analyzer.
- Backpatching and optimization are included as theoretical stages, not as fully implemented compiler phases.
- The application requires Python 3.10 or higher installed locally. It does not have a permanent production deployment.
- The local HTTP server is educational in nature and does not robustly support multiple simultaneous sessions.

22. Future Work

Future improvements may include: extending the grammar to support assignments, arithmetic expressions, multiple statements with values, **else**, **return**, and variable declarations; implementing a complete symbol table with type checking and scope handling; adding formal backpatching lists; implementing optimization passes over TAC; migrating to SLR(1), LR(1), or LALR for larger grammars; and deploying the application to a cloud server to eliminate the local Python installation dependency.

23. Conclusions

CompyFI LR(0) fulfills its didactic purpose by integrating the main front-end stages of a compiler into a visual web application. The project shows how source code is transformed into tokens, how an LR(0) parser validates the token sequence using states and an ACTION/GOTO table, how SDT concepts connect syntax with translation, and how elementary TAC can represent recognized structures.

The main contribution of the project is the visualization of the process. Instead of limiting itself to showing whether an input is accepted or rejected, CompyFI exposes each automaton state, each step of the trace, the symbol table, and the generated intermediate code, turning an otherwise abstract theory into an observable and understandable process.

From a technical perspective, the project demonstrated that a well-designed context-free grammar allows implementing an LR(0) parser without conflicts, and that the modular separation between lexical, syntactic, semantic, and TAC phases facilitates

both development and comprehension of the system. The fact that everything was implemented without external parsing libraries validates that the theoretical concepts of the course are sufficient to build a functional compiler.

Finally, CompyFI is not a production compiler, but it is a useful educational platform. It connects formal language theory, parser construction, syntax-directed translation, and intermediate code generation in a compact and understandable implementation.

References

- [1] R. A. Davila Perez, “Compiladores: apuntes de clase proporcionados en pizarron,” Facultad de Ingenieria, Universidad Nacional Autonoma de Mexico, semester 2026-2, 2026.
- [2] L. P. Arellano Mendoza, “Lenguajes Formales y Automatas: Introduccion y Conceptos,” class slides, n.d.
- [3] L. P. Arellano Mendoza, “Expresiones Regulares y Lenguajes,” class slides, n.d.
- [4] L. P. Arellano Mendoza, “Gramaticas Regulares y Automatas Finitos,” class slides, n.d.
- [5] L. P. Arellano Mendoza, “Gramaticas de Contexto Libre,” class slides, n.d.
- [6] R. A. Davila Perez, “Compiladores: analisis sintactico y resolucion de ambigüedad,” class notes provided on blackboard, Facultad de Ingenieria, Universidad Nacional Autonoma de Mexico, semester 2026-2, 2026.
- [7] L. P. Arellano Mendoza, “Automatas Push Down,” class slides, n.d.
- [8] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
- [9] J. E. Hopcroft, R. Motwani and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Pearson, 2006.

A. Project File Structure

The project repository follows the naming convention specified in the project requirements:

- **Package name:** `unam.fi.compilers.g5.08`
- **Repository:** Public with all source files.
- **Report:** `08-Compilers-FinalProject.pdf`

File / Folder	Description
main.py	Entry point and local HTTP server
ui.py	Web interface and rendering logic
lexer.py	Lexical analyzer
parser_lr0.py	LR(0) parser, canonical collection, ACTION/GOTO table
sdt.py	Semantic validation and symbol table
tac.py	TAC intermediate code generator
multimedia/	Interface resources
index.html	Initial HTML shell

B. Predefined Examples in the Application

ID	Title	Expected result
valid	Block with while and if	Accept — generates full TAC
empty	Empty block	Accept — no TAC
missing_semicolon	Missing semicolon	Syntactic error
incomplete_condition	Incomplete condition	Syntactic error
missing_brace	Missing closing brace	Syntactic error

C. Detailed Execution Instructions

1. Download or clone the project repository.
2. Verify that Python 3.10 or higher is installed: `python -version`
3. Open a terminal in the project root folder.
4. Run: `python main.py`
5. Open in the browser: `http://127.0.0.1:8000/index.html`
6. To stop the server: press **Ctrl+C** in the terminal.

Note: If running `python` produces an error, verify that Python is added to the system PATH. On Windows, reinstall Python and check the option “*Add Python to PATH*”.